

高层服务资源也成为 OS 管理对象，这是对系统之上应用层的扩展，例如，Internet 搜索和桌面搜索功能，已成为 OS 的基本功能之一。

(4) 支持功能、性能和可信性指标动态演化。对运算结果的控制要综合考虑能源消耗、运算精度以及快速处理等三个方面的要求之后，进行折中处理。

(5) 可信赖操作系统。开放环境所要求的整体化、多层次的可信性。构造可信赖系统包括私密性、防卫性、可靠性、安全性以及可用性。人们对信息安全给予空前的关注，这就要求 OS 能够支持更高强度的安全性能。

总之，软硬件之间的异构性促进了软件技术的发展。为屏蔽计算机硬件的异构性，发展了操作系统；为屏蔽操作系统和编程语言的异构性，发展了支撑软件和中间件；为了弥补应用软件与开发平台之间的距离，发展了软件框架、设计模式等技术。

3.6.2 进程管理

1. 程序与进程

1) 前驱图与前驱关系

前趋图是一个有向无循环图，由结点和有向边组成，结点代表各程序段的操作，而结点间的有向边表示两个程序段操作之间存在的前趋关系 ($\rightarrow$)。程序段 P_i 和 P_j 的前趋关系表示成 $P_i \rightarrow P_j$ ，其中， P_i 是 P_j 的直接前趋， P_j 是 P_i 的直接后继，其含义是 P_i 执行结束后 P_j 才能执行。程序顺序执行时的主要特征包括顺序性、封闭性和可再现性。

前驱关系 “ $\rightarrow$ ” 可形式化地记为 $\rightarrow = \{ \langle P_i, P_j \rangle \mid P_i \text{ 必须在 } P_j \text{ 执行前完成} \}$

【例 1】某系统中有一个 CPU、一台输入设备和一台输出设备，每个作业具有 3 个程序段，输入 I_i 、计算 C_i 和输出 P_i ($i=1,2,3$)。三个作业的各程序段并发执行的前驱图如图 3-11 所示。请给出该图的前驱关系。

【解】从图 3-11 中可以看出， I_2 与 C_1 并行执行； I_3 、 C_2 与 P_1 并行执行； C_3 与 P_2 并行执行。其中， I_2 、 I_3 受到 I_1 的间接制约， C_2 、 C_3 受到 C_1 的间接制约， P_2 、 P_3 受到 P_1 的间接制约，而 C_1 、 P_1 受到 I_1 的直接制约，等等。

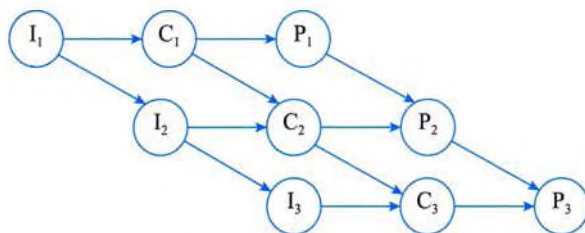


图 3-11 程序并发执行的前驱图

经上分析，图 3-11 中的前驱关系可记为：

$$\rightarrow = \{ \langle I_1, C_1 \rangle, \langle I_1, I_2 \rangle, \langle I_2, C_2 \rangle, \langle I_2, I_3 \rangle, \langle I_3, C_3 \rangle, \langle C_1, P_1 \rangle, \langle C_1, C_2 \rangle, \langle C_2, P_2 \rangle, \langle C_2, C_3 \rangle, \langle C_3, P_3 \rangle, \langle P_1, P_2 \rangle, \langle P_2, P_3 \rangle \}$$

或者也可以记为如下形式：

$$\rightarrow = \{I_1 \rightarrow C_1, I_1 \rightarrow I_2, I_2 \rightarrow C_2, I_2 \rightarrow I_3, I_3 \rightarrow C_3, C_1 \rightarrow P_1, C_1 \rightarrow C_2, C_2 \rightarrow P_2, C_2 \rightarrow C_3, \\ C_3 \rightarrow P_3, P_1 \rightarrow P_2, P_2 \rightarrow P_3\}$$

程序并发执行的特征包括三方面：第一，失去了程序的封闭性；第二，程序和机器执行程序的活动不再一一对应；第三，并发程序间的相互制约性。

2) 进程的组成

进程是程序的一次执行，该程序可以和其他程序并发执行。进程通常是由进程控制块（Process Control Block, PCB）、程序和数据组成的。

(1) PCB。PCB 是进程存在的唯一标志。

(2) 程序。程序部分描述了进程需要完成的功能。假如一个程序能被多个进程同时共享执行，那么这一部分就应该以可再入（纯）码的形式编制，它是程序执行时不可修改的部分。

(3) 数据。数据部分包括程序执行时所需的数据及工作区。该部分只能为一个进程专用，是进程的可修改部分。

2. 进程的状态与状态模型图

1) 三态模型

在多道程序系统中，进程在处理器上交替运行，状态也不断地发生变化，因此进程一般有三种基本状态：运行、就绪和阻塞。进程的三态模型如图 3-12 所示。

(1) 运行。当一个进程在处理器上运行时，则称该进程处于运行状态。显然，对于单处理机系统，处于运行状态的进程只有一个。

(2) 就绪。一个进程获得了除处理机外的一切所需资源，一旦得到处理机即可运行，则称此进程处于就绪状态。

(3) 阻塞。阻塞也称等待或睡眠状态，一个进程正在等待某一事件发生（例如，请求 I/O、等待 I/O 完成等）而暂时停止运行，这时即使把处理机分配给进程也无法运行，故称该进程处于阻塞状态。

2) 五态模型

事实上，对于一个实际的系统，进程的状态及其转换更复杂。例如，引入新建态和终止态，构成了进程的五态模型，如图 3-13 所示。

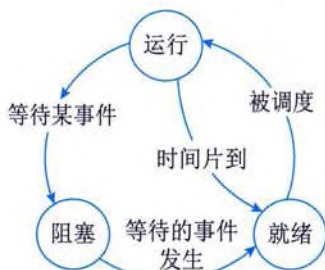


图 3-12 进程的三态模型

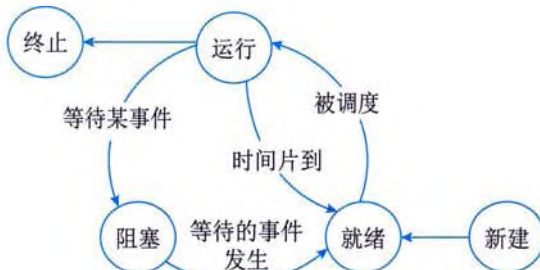


图 3-13 进程的五态模型

在图 3-13 中，新建态对应于进程刚刚被创建时没有被提交的状态，并等待系统完成创建进程的所有必要信息。因为创建进程时分为两个阶段，第一个阶段为一个新进程创建必要的管理信息，第二个阶段让该进程进入就绪状态。由于有了新建态操作系统，往往可以根据系统的性能和主存容量的限制推迟新建态进程的提交。

类似地，进程的终止也可分为两个阶段，第一个阶段等待操作系统进行善后处理，第二个阶段释放主存。

3. 信号量与 PV 操作

荷兰学者 Dijkstra 于 1965 年提出的信号量机制是一种有效的进程同步与互斥工具。目前，信号量机制有了很大的发展，主要有整型信号量、记录型信号量和信号量集机制。

1) 整型信号量与 PV 操作

信号量是一个整型变量，根据控制对象的不同被赋予不同的值。信号量分为如下两类：

- (1) 公用信号量。实现进程间的互斥，初值为 1 或资源的数目。
- (2) 私用信号量。实现进程间的同步，初值为 0 或某个正整数。

信号量 S 的物理意义： $S \geq 0$ 表示某资源的可用数；若 $S < 0$ ，则其绝对值表示阻塞队列中等待该资源的进程数。

对于系统中的每个进程，其工作的正确与否不仅取决于它自身的正确性，而且与它在执行中能否与其他相关进程正确地实施同步与互斥有关。PV 操作是实现进程同步与互斥的常用方法。P 操作和 V 操作是低级通信原语，在执行期间不可分割。其中，P 操作表示申请一个资源，V 操作表示释放一个资源。

P 操作的定义： $S := S - 1$ ，若 $S \geq 0$ ，则执行 P 操作的进程继续执行；若 $S < 0$ ，则置该进程为阻塞状态（因为无可用资源），并将其插入阻塞队列。P 操作可用如下过程表示。

```

Procedure P(Var S:Semaphore);    // Semaphore 表示所定义的变量是信号量
Begin
    S:=S-1;
    If S<0 then W(S)              {执行 P 操作的进程插入等待队列}
End;
```

V 操作的定义： $S := S + 1$ ，若 $S > 0$ ，则执行 V 操作的进程继续执行；若 $S \leq 0$ ，则从阻塞状态唤醒一个进程，并将其插入就绪队列，然后执行 V 操作的进程继续。

V 操作可用如下过程表示。

```

Procedure V(Var S:Semaphore);
Begin
    S:=S+1;
    If S≤0 then R(S)              {从阻塞队列中唤醒一个进程}
End;
```

2) 利用 PV 操作实现进程的互斥

令信号量 `mutex` 的初值为 1，当进入临界区时执行 P 操作，退出临界区时执行 V 操作。这

样，利用 PV 操作实现进程互斥的代码段如下：

P(mutex)

临界区

V(mutex)

3) 利用 PV 操作实现进程的同步

进程的同步是由于进程间合作引起的相互制约的问题，要实现进程的同步可用一个信号量与消息联系起来，当信号量的值为 0 时表示希望的消息未产生，当信号量的值为非 0 时表示希望的消息已经存在。假定用信号量 S 表示某条消息，进程可以通过调用 P 操作测试消息是否到达，调用 V 操作通知消息已准备好。最典型的同步问题是缓冲区的生产者和消费者的同步问题。

【例 2】一个生产者和一个消费者，缓冲区中可存放 n 件产品，生产者不断地生产产品，消费者不断地消费产品，如何用 PV 操作实现生产者和消费者的同步。

【解】本题可以通过设置 3 个信号量 S 、 S_1 和 S_2 来解决。其中：

S 是一个互斥信号量，初值为 1。由于缓冲区是一个互斥资源，所以需要进行互斥控制。

S_1 是一个同步信号量，初值为 n ，表示缓冲区的空闲单元数，即是否可以将产品放入缓冲区。

S_2 是一个同步信号量，表示缓冲区是否存有产品，初值为 0。

综上所述，一个生产者、一个消费者的同步过程如图 3-14 所示。

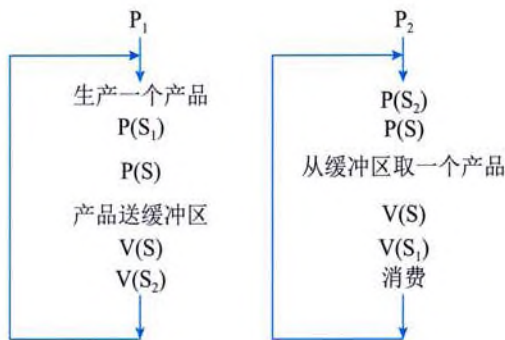


图 3-14 n 个缓冲区的同步举例

4. 高级通信原语

进程间通信是指进程之间的信息交换，少则一个信息，多则成千上万个信息。根据交换信息量的多少和效率的高低，进程通信的方式分为低级方式和高级方式。PV 操作属于低级通信方式，若用 PV 操作实现进程间通信，则存在如下问题：

(1) 编程难度大。通信对用户不透明，即要用户利用低级通信工具实现进程间的同步与互斥。而且，PV 操作使用不当容易引起死锁。

(2) 效率低。在上例中，生产者每次只能向缓冲区送一个产品，消费者每次只能从缓冲区取一个产品。

为了提高信号通信的效率，传递大量数据，降低程序编制的复杂度，系统引入了高级通信方式。高级通信方式主要分为共享存储模式、消息传递模式和管道通信。

(1) 共享存储模式。相互通信的进程可以共享某些数据结构（或存储区）实现进程之间的通信。

(2) 消息传递模式。进程间的数据交换以消息为单位，程序员直接利用系统提供的一组通信命令（原语）来实现通信，如 Send(A)、Receive(A)。

(3) 管道通信。所谓管道，是指用于连接一个读进程和一个写进程，以实现它们之间通信的共享文件（pipe 文件）。向管道（共享文件）提供输入的发送进程（即写进程），以字符流的形式将大量的数据送入管道；而接收进程可从管道接收大量的数据。由于它们通信时采用管道，所以称为管道通信。

5. 产生死锁的原因及处理策略

1) 产生死锁的原因

死锁是指两个以上的进程互相都要求使用对方已经占有的资源而导致无法继续运行下去的现象。产生死锁的主要原因是，系统缺少足够的资源供进程使用，对互斥资源的共享与并发执行的顺序不当，以及资源分配不当。产生死锁的4个必要条件如下：

- (1) 互斥条件。任一时刻只允许一个进程使用资源。
- (2) 不剥夺条件。进程已经占用的资源，不会被强制剥夺。
- (3) 请求与保持条件。进程在请求其余资源时，不主动释放已经占有的资源。
- (4) 环路条件。环路中每一条边是进程在请求另一个进程已经占有的资源。

2) 死锁的处理策略

死锁的处理策略主要有4种：鸵鸟策略（即不理睬策略）、预防策略、避免策略和检测与解除死锁。

(1) 死锁预防。死锁预防是采用某种策略限制并发进程对资源的请求，破坏死锁产生的4个必要条件之一，使系统在任何时刻都不满足死锁的必要条件。预防死锁的两种策略如下：

- 预先静态分配法。破坏了“不可剥夺条件”，预先分配所需资源，保证不等待资源。该方法的问题是降低了对资源的利用率，降低进程的并发程度；有时可能无法预先知道所需资源。
- 资源有序分配法。破坏了“环路条件”，把资源分类按顺序排列，保证不形成环路。该方法存在的问题是限制进程对资源的请求；因资源排序占用系统开销。

(2) 死锁避免。死锁避免是设法破坏产生死锁的4个必要条件之一，严格防止死锁的产生。死锁避免则不那么严格地限制产生死锁的必要条件。最著名的死锁避免算法是 Dijkstra 提出的银行家算法，死锁避免算法需要很大的系统开销。

银行家算法对于进程发出的每一个系统可以满足的资源请求命令加以检测，如果发现分配资源后系统进入不安全状态，则不予分配；若分配资源后系统仍处于安全状态，则实施分配。

与死锁预防策略相比，它提高了资源的利用率，但检测分配资源后系统是否安全增加了系统开销。安全状态是指系统能按某种顺序，如 $\langle P_1, P_2, \dots, P_n \rangle$ 来为每个进程分配其所需资源，直到最大需求，使每个进程都可顺序完成，则称 $\langle P_1, P_2, \dots, P_n \rangle$ 序列为安全序列；若系统不存在这样一个安全序列，则称系统处于不安全状态。

3) 死锁检测与解除

解决死锁的另一条途径是使用死锁检测方法，这种方法对资源的分配不加限制，即允许死

锁产生。但系统定时地运行一个死锁检测程序，判断系统是否发生死锁，若检测到有死锁，则设法加以解除。死锁解除通常采用资源剥夺法和撤销进程法。资源剥夺法是从一些进程那里强行剥夺足够数量的资源分配给死锁进程；撤销进程法是根据某种策略逐个地撤销死锁进程，直到解除死锁为止。

6. 线程

传统的进程有两个基本属性：可拥有资源的独立单位；可独立调度和分配的基本单位。引入线程的原因是进程在创建、撤销和切换中，系统必须为之付出较大的时空开销，故在系统中设置的进程数目不宜过多，进程切换的频率不宜太高。

引入线程后，将传统进程的两个基本属性分开，线程作为调度和分配的基本单位，进程作为独立分配资源的单位。用户可以通过创建线程来完成任务，以减少程序并发执行时付出的时空开销。例如，在文件服务进程中可设置多个服务线程，当一个线程受阻时，第二个线程可以继续运行，当第二个线程受阻时，第三个线程可以继续运行……从而显著地提高文件系统的服务质量及系统的吞吐量。这样，对于拥有资源的基本单位，不用频繁地切换，进一步提高了系统中各程序的并发程度。

需要说明的是，线程是进程中的一个实体，是被系统独立分配和调度的基本单位。线程基本上不拥有资源，只拥有一点运行中必不可少的资源（如程序计数器、一组寄存器和栈），它可与同属一个进程的其他线程共享进程所拥有的全部资源。

线程也具有就绪、运行和阻塞3种基本状态。由于线程具有许多传统进程所具有的特性，故称为“轻型进程（Light-Weight Process）”。线程可创建另一个线程，同一个进程中的多个线程可并发执行。线程分为用户级线程（User-Level Threads）和内核支持线程（Kernel-Supported Threads）两类。用户级线程不依赖于内核，该类线程的创建、撤销和切换都不利用系统调用来实现；内核支持线程依赖于内核，即无论是在用户进程中的线程，还是在系统中的线程，它们的创建、撤销和切换都利用系统调用来实现。

3.6.3 存储器管理

存储器管理的对象是主存存储器，简称为主存或内存。存储器是计算机系统中的关键性资源，是存放各种信息的主要场所。如何对存储器实施有效的管理，不仅直接影响存储器的利用率，而且还对系统性能有很大的影响。存储器管理的主要功能包括主存空间的分配和回收、提高主存的利用率、扩充主存、对主存信息实现有效保护。

存储器管理的主要目的是解决多个用户使用主存的问题，其存储管理方案主要包括分区存储管理、分页存储管理、分段存储管理、段页式存储管理以及虚拟存储管理。

1. 分区存储管理

分区存储管理是早期的存储管理方案，其基本思想是把主存的用户区划分成若干个区域，每个区域分配给一个用户作业使用，并限定它们只能在自己的区域中运行。

按划分方式不同，分区存储管理可分为固定分区、可变分区和可重定位分区。其中，固定分区是一种静态分区方式；可变分区是一种动态分区方式；可重定位分区的基本思想是移动所

有已分配好的分区，使之成为连续区域，以减少内存中的碎片。

2. 分页存储管理

在分页存储管理中，地址结构由两部分组成：页号 P 和页内地址 W 。系统为每个进程建立了一张页面映射表（简称页表），记录进程的每个页面对应的物理块号。

分页存储管理的地址变换机构如图 3-15 所示，其任务是利用页表把用户程序中的逻辑地址变换成主存中的物理地址。

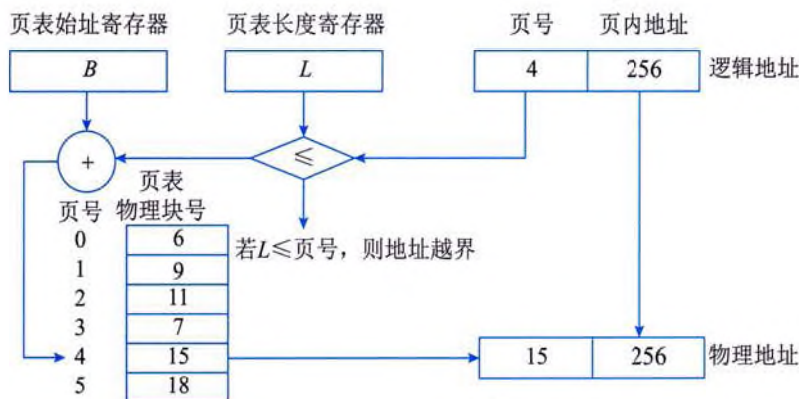


图 3-15 分页存储管理的地址变换机构

在进行地址变换时，系统将页号（页号从 0 开始）与页表长度寄存器中的 L 进行比较，若页号大于等于 L ，则访问越界，产生越界中断；若未越界，则根据页表寄存器中的页表起始地址和页号计算出该页在页表项中的位置，得到该页的物理块号，将此物理块号装入物理地址寄存器中。与此同时，将有效地址（逻辑地址）寄存器中的页内地址直接装入物理地址寄存器的块内地址字段中，这样便完成了从逻辑地址到物理地址的变换。

进程在执行时，系统通过查找页表就可以找到每页所对应的物理块号。例如，图 3-15 中逻辑页号为 4，查找页表可得该页的物理块号为 15，与页内地址 256 拼接得到物理地址 15256。可见，页表的作用是实现从页号到物理块号的地址映射。

3. 分段存储管理

在分段存储管理方式中，作业的地址空间被划分为若干个段，每个段是一组完整的逻辑信息，例如有主程序段、子程序段、数据段及堆栈段等，每个段都有自己的名字，都是从 0 开始编址的一段连续的地址空间，各段的长度是不等的。分段系统的逻辑地址结构由段号（名）和段内地址两部分组成。

在分段存储管理系统中，为每个段分配一个连续的分区，而进程中的各个段可以离散地分配到主存的不同分区中。在系统中为每个进程建立一张段映射表，简称为段表。每个段在表中占有一个表项，在其中记录了该段在主存中的起始地址（又称为基址）和段的长度。进程在执行时，通过查段表来找到每个段所对应的主存区。可见，段表实现了从逻辑段到物理主存区的映射。分段存储管理的地址变换机构如图 3-16 所示。

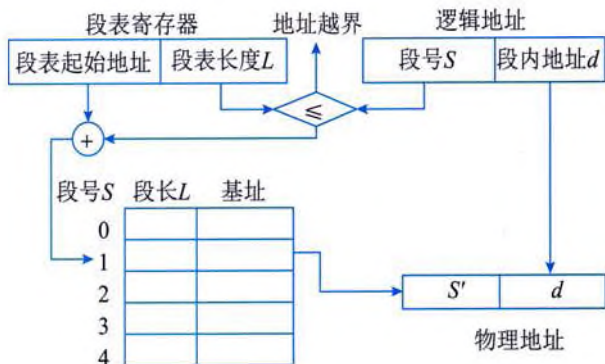


图 3-16 分段存储管理的地址变换机构

为了实现从逻辑地址到物理地址的变换功能，系统中设置了段表寄存器，用于存放段表起始地址和段表长度。在进行地址变换时，系统将逻辑地址中的段号 S 与段表长度 L 进行比较。若 $S \geq L$ ，表示段号太大，访问越界，产生越界中断信号；否则，根据段表的起始地址和该段的段号，计算出该段对应段表项的位置，从中读出该段在主存中的起始地址，然后再检查段内地址 d 是否超过该段的段长 SL 。若超过，即 $d \geq SL$ ，同样发出越界中断信号；否则，将该段的基址 S' 与段内地址 d 相加，得到要访问的主存物理地址。

4. 段页式存储管理

段页式存储管理的基本原理是先将整个主存划分成大小相等的存储块（页框），将用户程序按程序的逻辑关系分为若干个段，并为每个段赋予一个段名，再将每个段划分成若干页，以页框为单位离散分配。段页式存储管理的地址变换机构如图 3-17 所示。

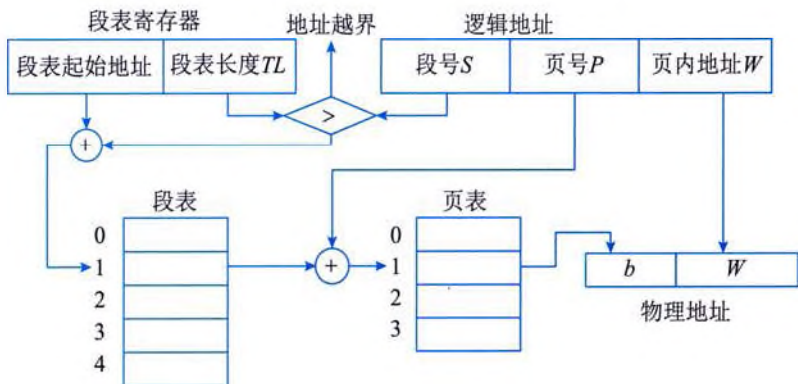


图 3-17 段页式存储管理的地址变换机构

地址结构由段号、段内页号和页内地址三部分组成。为了实现从逻辑地址到物理地址的变换，系统中必须同时配置段表和页表，段表中的内容是页表的起始地址和页表长度。

在段页式存储管理中，逻辑地址到物理地址的变换过程如下：

- (1) 根据段号 S 查段表，得到页表的起始地址；
- (2) 根据页号 P 查页表，得到物理块号 b ；

(3) 将物理块号 b 拼页内地址 W 得到物理地址。

5. 虚拟存储管理

在前面介绍的存储管理方案中，必须为每个作业分配足够的空间，以便装入全部信息。当主存空间不能满足作业要求时，作业无法装入主存执行。

如果一个作业只部分装入主存便可开始启动运行，其余部分暂时留在磁盘上，在需要时再装入主存，这样可以有效地利用主存空间。从用户角度看，该系统所具有的主存容量将比实际主存容量大得多，人们把这样的存储器称为虚拟存储器。虚拟存储器是为了扩大主存容量而采用的一种设计方法，其容量是由计算机的地址结构决定的。

1) 程序的局限性原理

早在 1968 年 P. Denning 就指出，程序在执行时将呈现出局部性规律，即在一段时间内，程序的执行仅局限于某个部分。相应地，它所访问的存储空间也局限于某个区域内。程序的局限性表现在时间局限性和空间局限性两个方面。

(1) 时间局限性是指如果程序中的某条指令一旦执行，则不久的将来该指令可能再次被执行；如果某个存储单元被访问，则不久以后该存储单元可能再次被访问。产生时间局限性的典型原因是，在程序中存在大量的循环操作。

(2) 空间局限性是指一旦程序访问了某个存储单元，则在不久的将来，其附近的存储单元也最有可能被访问，即程序在一段时间内所访问的地址可能集中在一定的范围内，其典型原因是程序的顺序执行。

2) 工作集理论

事实上，程序在运行中所产生的缺页情况会影响程序的运行速度及系统性能，而缺页率的高低又与每个进程所占用的物理块数目有关。但是，究竟应该为每个进程分配多少个物理块才能把缺页率保持在一个合理的水平上是无法预测的，否则会因为进程频繁地从辅存请求页面而出现“颠簸”（也称抖动）现象。为了解决这一问题，引入了工作集理论。

工作集的理论是在 1968 年由 P. Denning 提出的，他认为，虽然程序只需有少量的几页在主存就可以运行，但为了使程序能够有效地运行，较少地产生缺页，必须使程序的工作集驻留在主存中。把某进程在时间 t 的工作集记为 $w(t, \Delta)$ ，变量 Δ 称为工作集窗口尺寸。正确地选择工作集窗口 (Δ) 的大小，对存储器的有效利用和系统吞吐量的提高都将产生重大的影响。可见，工作集就是指在某段时间间隔 (Δ) 里，进程实际要访问的页面的集合。

需要注意的是，程序在运行时对页面的访问是不均匀的，如果能够预知程序在某段时间间隔内要访问哪些页面，并能将它们提前调入主存，将会大大地降低缺页率，从而减少置换工作，提高 CPU 的利用率。当每个工作集都已达到最小值时，虚存管理程序跟踪进程的缺页数量，根据主存中自由页面的数量可以适当增加其工作集的大小。

3) 虚拟存储器的实现

虚拟存储器是具有请求调入功能和置换功能，仅把作业的一部分装入主存便可运行作业的存储器系统，是能从逻辑上对主存容量进行扩充的一种虚拟的存储器系统。其逻辑容量由主存和外存容量之和以及 CPU 可寻址的范围来决定，其运行速度接近于主存速度，成本也有所下

降。可见，虚拟存储技术是一种性能非常优越的存储器管理技术，故被广泛地应用于大、中、小型机器和微型机中。虚拟存储器的实现主要有如下3种方式：

(1) 请求分页系统。该系统是在分页系统的基础上增加了请求调页功能和页面置换功能所形成的页式虚拟存储系统。它允许只装入若干页的用户程序和数据（而非全部程序）就可以启动运行，以后再通过调页功能和页面置换功能陆续把将要使用的页面调入主存，同时把暂不运行的页面置换到外存上，置换时以页面为单位。

(2) 请求分段系统。该系统是在分段系统的基础上增加了请求调段功能和分段置换功能所形成的段式虚拟存储系统。它允许只装入若干段的用户程序和数据就可以启动运行，之后再通过调段功能和分段置换功能将不运行的段调出，同时调入将要运行的段。置换时以段为单位。

(3) 请求段页式系统。该系统是在段页式系统的基础上增加了请求调页功能和页面置换功能所形成的段页式虚拟存储系统。

4) 页面置换算法

请求分页是在纯分页系统的基础上增加了请求调页功能、页面置换功能所形成的页式虚拟存储系统，它是目前常用的一种虚拟存储器的方式。在进程运行过程中，如果发生缺页，此时主存中又无空闲块时，为了保证进程能正常运行，必须从主存中调出一页程序或数据送到磁盘的对换区。但究竟将哪个页面调出，需要根据一定的页面置换算法来确定。置换算法的好坏将直接影响系统的性能，不适当的算法可能会导致系统发生“抖动”（Thrashing），即刚被换出的页很快又被访问，需重新调入，导致系统频繁地更换页面，以至于一个进程在运行中把大部分时间花费在完成页面置换的工作上，这种现象称为系统发生了“抖动”（也称颠簸）。请求分页系统的核心问题是选择合适的页面置换算法。常用的页面置换算法如下所述：

(1) 最佳（Optimal）置换算法。这是一种理想化的算法，即选择那些永不使用的，或者是在最长时间内不再被访问的页面置换出去。这种方法性能最好，但实际上难以实现，并且要确定哪一个页面是未来最长时间内不再被访问的是很难的，所以该算法通常用来评价其他算法。

(2) 先进先出（FIFO）置换算法。该算法总是淘汰最先进入主存的页面，即选择在主存中驻留时间最久的页面予以淘汰。该算法实现简单，只需把一个进程调入主存的页面，按先后次序链接成一个队列，并设置一个指针即可。FIFO是一种最直观、性能最差的算法，并且有Belady异常现象。所谓Belady现象，是指如果对一个进程未分配它所要求的全部页面，有时就会出现分配的页面数增多但缺页率反而提高的异常现象。

(3) 最近最少使用（Least Recently Used, LRU）置换算法。LRU算法是选择最近最少使用的页面予以淘汰，系统在每个页面设置一个访问字段，用于记录这个页面自上次被访问以来所经历的时间 T ，当要淘汰一个页面时，选择 T 最大的页面，但在实现时需要硬件的支持（寄存器或栈）。

(4) 最近未用（Not Used Recently, NUR）置换算法。NUR算法将最近一段时间未引用过的页面换出，这是一种LRU的近似算法。该算法为每个页面设置一位访问位，将主存中的所有页面都通过链接指针链成一个循环队列。当某页被访问时，其访问位置1。在选择一页淘汰时，检查其访问位，如果是0，则选择该页换出；若为1，则重新置为0，暂不换出该页，在循环

队列中检查下一个页面，直到访问位为0的页面为止。由于该算法只有一位访问位，只能用它表示该页是否已经使用过，而置换时是将未使用过的页面换出去，所以把该算法称为最近未用算法。

3.6.4 设备管理

在计算机系统中，将负责管理设备和输入/输出的机构称为I/O系统。I/O系统由设备、控制器、通道（具有通道的计算机系统）、总线和I/O软件组成。

设备管理的目标主要是提高设备的利用率，为用户提供方便、统一的界面。提高设备的利用率，就是提高CPU与I/O设备之间的并行操作程度。在设备管理中，主要利用的技术有中断技术、DMA技术、通道技术和缓冲技术。

1. I/O 软件

I/O设备管理软件一般分为4层：中断处理程序、设备驱动程序、与设备无关的系统软件 and 用户级软件。至于一些具体分层时细节上的处理，是依赖于系统的，没有严格的划分，只要有利于设备独立这一目标，就可以为了提高效率设计不同的层次结构。

I/O软件的所有层次及每一层的主要功能如图3-18所示，图中的箭头给出了I/O部分的控制流。



图 3-18 I/O 系统的层次结构及各层的主要功能

例如，读一个硬盘上的文件。用户程序需要通过操作系统实现这一操作，其工作步骤为：与设备无关软件检查高速缓存中是否有要读的数据块，若没有，则调用设备驱动程序，向I/O硬件发出一个请求；用户进程阻塞并等待磁盘操作的完成；当磁盘操作完成时，硬件产生一个中断，转入中断处理程序；中断处理程序检查中断的原因，认识到这时磁盘读取操作已经完成，于是唤醒用户进程取回从磁盘读取的信息，从而结束此次I/O请求；用户进程在得到了所需的硬盘文件内容之后，继续运行。

2. 设备管理采用的关键技术

设备管理采用的关键技术包括通道技术、DMA技术、缓冲技术和SPOOLing技术。通道技术和DMA技术参见3.3.1小节。本小节介绍SPOOLing技术。

SPOOLing（Simultaneous Peripheral Operations On Line）也称外围设备联机操作。实际上，SPOOLing技术是用一类物理设备模拟另一类物理设备，使得独占使用的设备变成多台虚拟设备的一种技术。SPOOLing系统的组成和结构如图3-19所示。

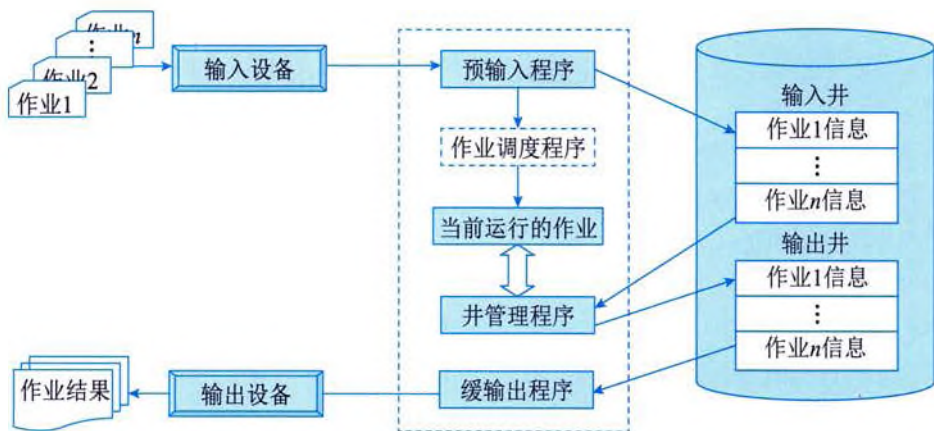


图 3-19 SPOOLing 系统的组成和结构

SPOOLing 系统是由“预输入程序”“缓输出程序”“井管理程序”以及输入和输出井组成的。其中，输入井和输出井是为了存放从输入设备输入的信息以及作业执行的结果，输入井和输出井是系统在辅助存储器上开辟的存储区域。

SPOOLing 系统的工作过程是：操作系统初启后激活 SPOOLing 预输入程序，使它处于捕获输入请求的状态，一旦有输入请求消息，SPOOLing 输入程序立即得到执行，把装在输入设备上的作业输入到硬盘的输入井中并填写好作业表，以便在作业执行中要求输入信息时可以随时找到它们的存放位置。当作业需要输出数据时，可以先将数据送到输出井，当输出设备空闲时，由 SPOOLing 输出程序把硬盘上输出井的数据送到慢速的输出设备上。

SPOOLing 系统中拥有一张作业表，用来登记进入系统的所有作业的作业名、状态和预输入表位置等信息。每个用户作业拥有一张预输入表来登记该作业的各个文件的情况，包括设备类、信息长度及存放位置等。

输入井中的作业有如下 4 种状态：

- (1) 提交状态。作业的信息正从输入设备上预输入。
- (2) 后备状态。作业预输入结束但未被选中执行。
- (3) 执行状态。作业已被选中运行，在运行过程中，它可从输入井中读取数据信息，也可向输出井写信息。
- (4) 完成状态。作业已经撤离，该作业的执行结果等待缓输出。

3. 磁盘调度

磁盘是可被多个进程共享的设备。当有多个进程请求访问磁盘时，为了保证信息的安全，系统在每一时刻只允许一个进程启动磁盘进行 I/O 操作，其余的进程只能等待。因此，操作系统应采用一种适当的调度算法，使各进程对磁盘的平均访问（主要是寻道）时间最小。磁盘调度分为移臂调度和旋转调度两类，并且是先进行移臂调度，然后进行旋转调度。由于访问磁盘最耗时的是寻道时间，因此，磁盘调度的目标是使磁盘的平均寻道时间最少。

常用的磁盘调度算法如下：

- (1) 先来先服务（First-Come First-Served, FCFS）。这是最简单的磁盘调度算法，它根据进